



```
public abstract class Animal { protected String nombre; public abstract void hacerSonido(); } public interface Volador { void volar(); }
```

</> Programación Orientada a Objetos

Java

Clases Abstractas e Interfaces en Java

Diseño flexible con ejemplos de animales.



Clases Abstractas

Estado y comportamiento base



Interfaces

Contratos de capacidad

```
Ejemplo.java

public abstract class Animal{

    public abstract void hacerSonido ();

}

public interface Volador {

    void volar();

}

public class Loro extends Animal implements Volador {

    @Override
    public void hacerSonido() {

    }

    @Override
    public void volar() {

    }

}
```



¿Qué aprenderemos hoy?

01

¿Por qué usar clases abstractas?

02

¿Qué es una interfaz?

03

Clase abstracta vs Interfaz

04

Clase abstracta Animal

05

Subclase Perro y Gato

06

Interfaces en Acción

07

Implementación múltiple

08

Combinación Clases abstractas e interfaz

09

Polimorfismo con clases abstractas e interfaces

10

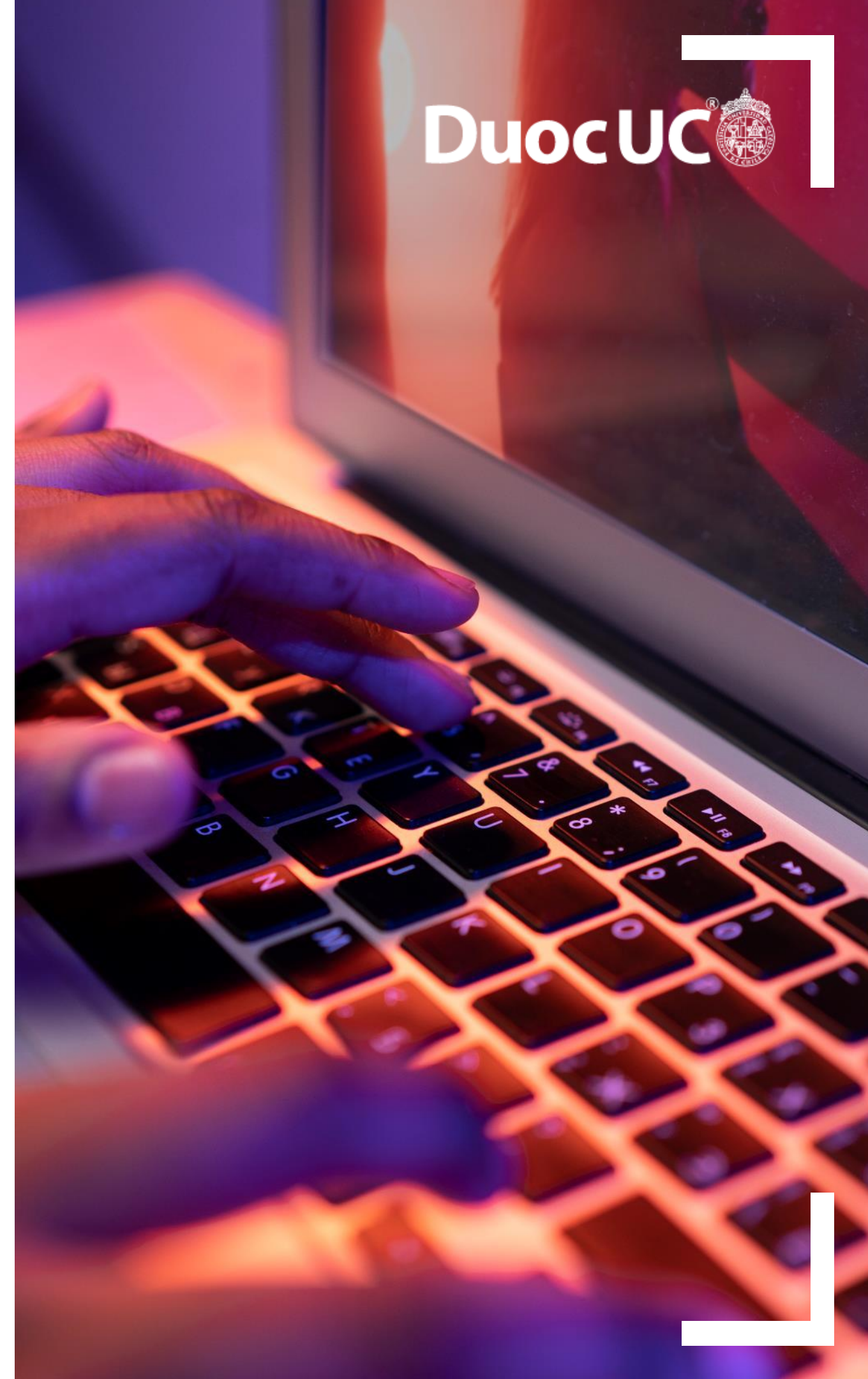
Buenas practicas de diseño

11

Errores Comunes

12

Conclusión y próximos pasos



¿Qué es una Clase Abstracta?

Base común que no se puede instanciar directamente

Definición y Claves

Es una plantilla parcial en POO. Sirve como **base obligatoria** para otras clases, pero no puede instanciarse por sí misma (ej. `new Animal()` da error).



No Instanciable

Previene la creación de objetos genéricos sin sentido.



Base Común

Agrupar estado (atributos) y comportamiento compartido.



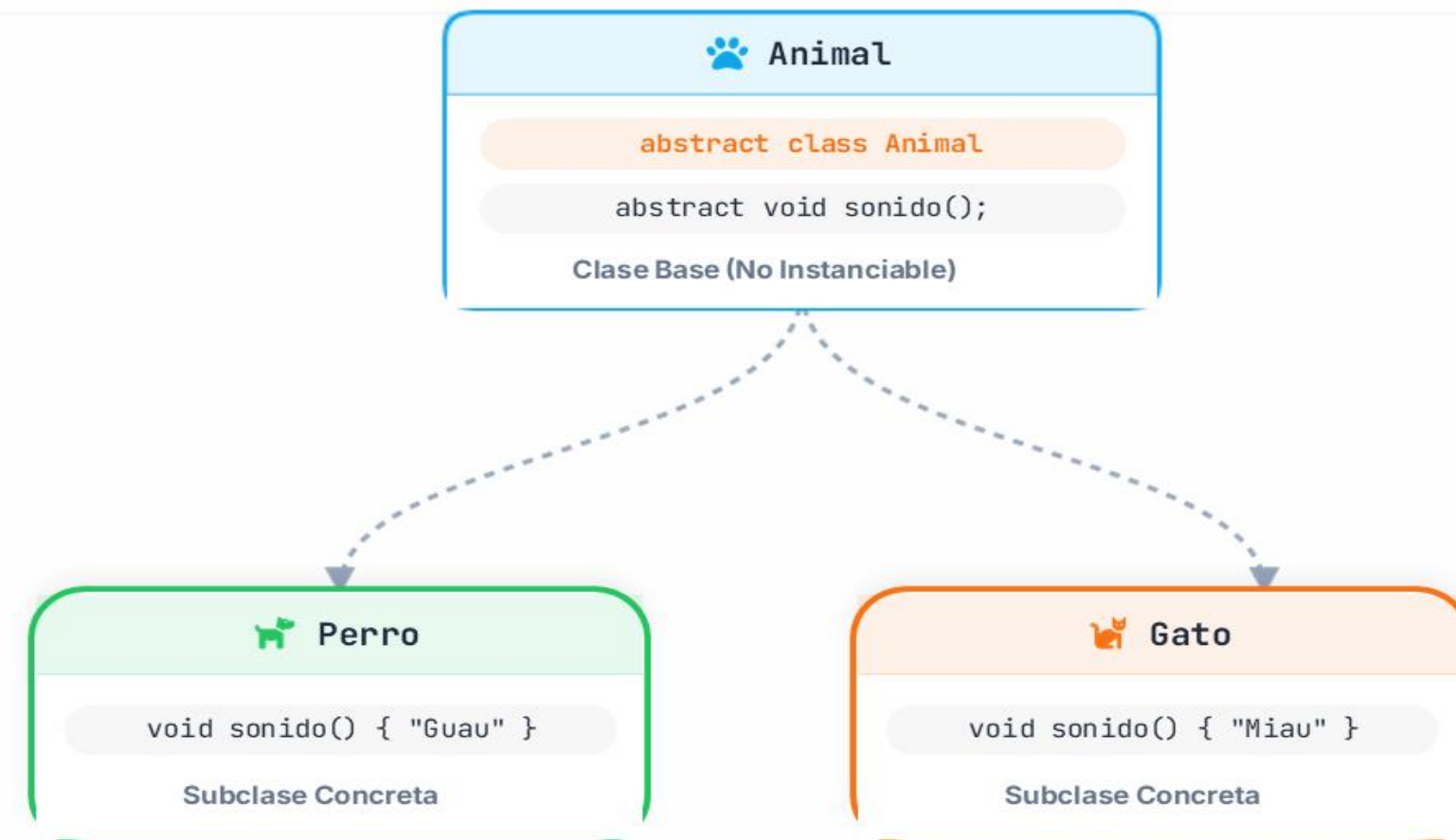
Contrato Parcial

Puede combinar métodos con código y métodos abstractos.

Regla de Oro:

- Una clase con al menos un método abstracto, debe ser abstracta.

Estructura Jerárquica: Clase Abstracta



 No se instancia directamente

 Fuerza la implementación en subclases



¿Por qué usar clases abstractas?

Diseño base, lógica compartida y contratos en Java

★ Beneficios Principales

Las clases abstractas definen una base estructural sólida, combinando código ya implementado con métodos obligatorios para sus subclases.



Lógica Común

Reutiliza estado y comportamiento idéntico (ej. dormir, edad).



Contrato Parcial

Fuerza a las subclases a implementar comportamientos específicos.



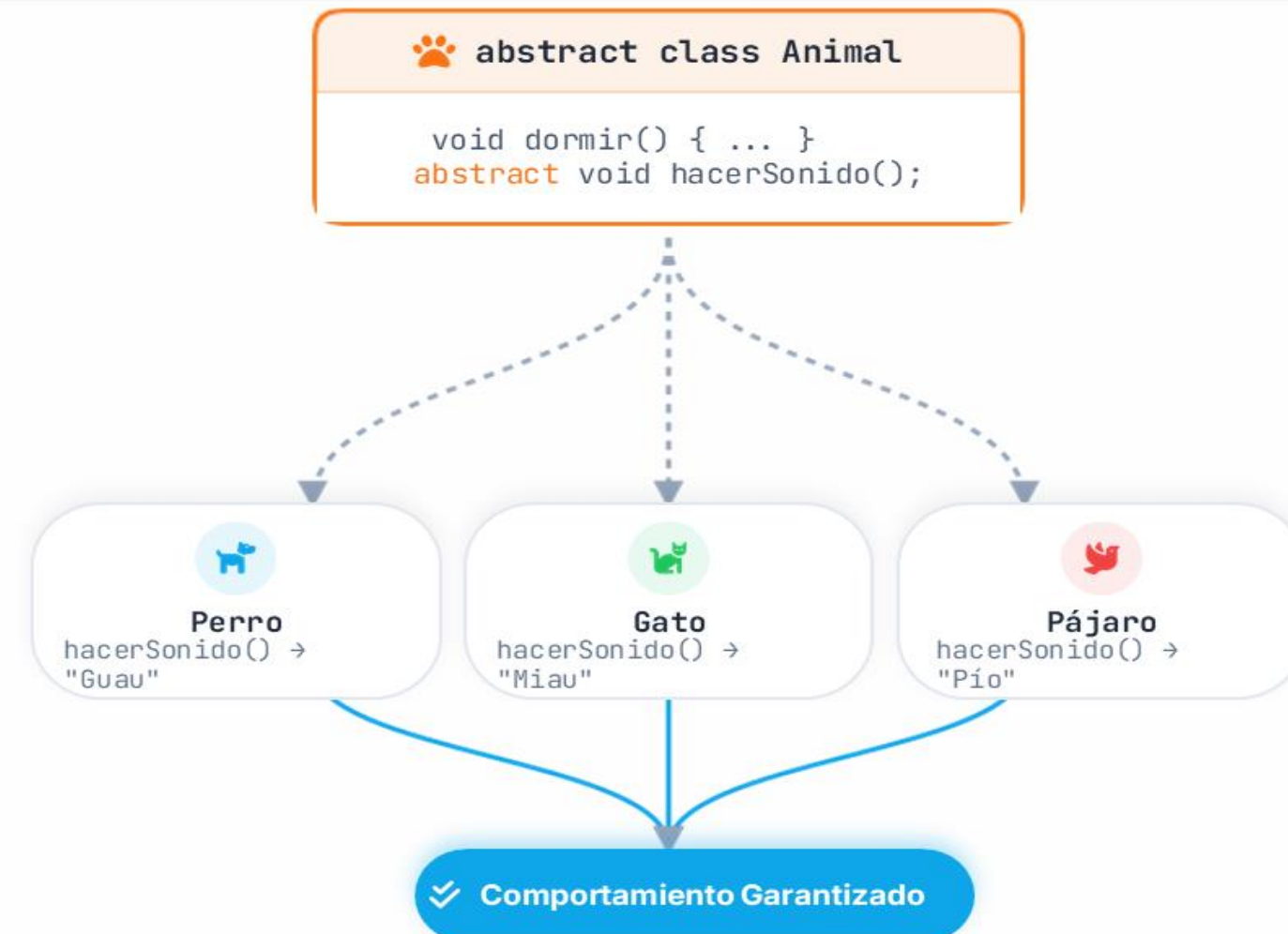
Especialización

Controla qué partes de la clase son fijas y cuáles son adaptables.

Resultado Final:

- ✓ Código estructurado y seguro

Flujo de Implementación



¿Qué es una interfaz?

Definiendo capacidades y contratos en Java

Conceptos Clave

Una interfaz es un contrato puramente abstracto que define qué debe hacer una clase, sin especificar cómo debe hacerlo. Modela "capacidades".



Contrato Estricto

Solo define las firmas de los métodos. Las clases que la implementan proveen el código.



Capacidad: Volador

Cualquier clase que implemente Volador debe proporcionar el método volar().



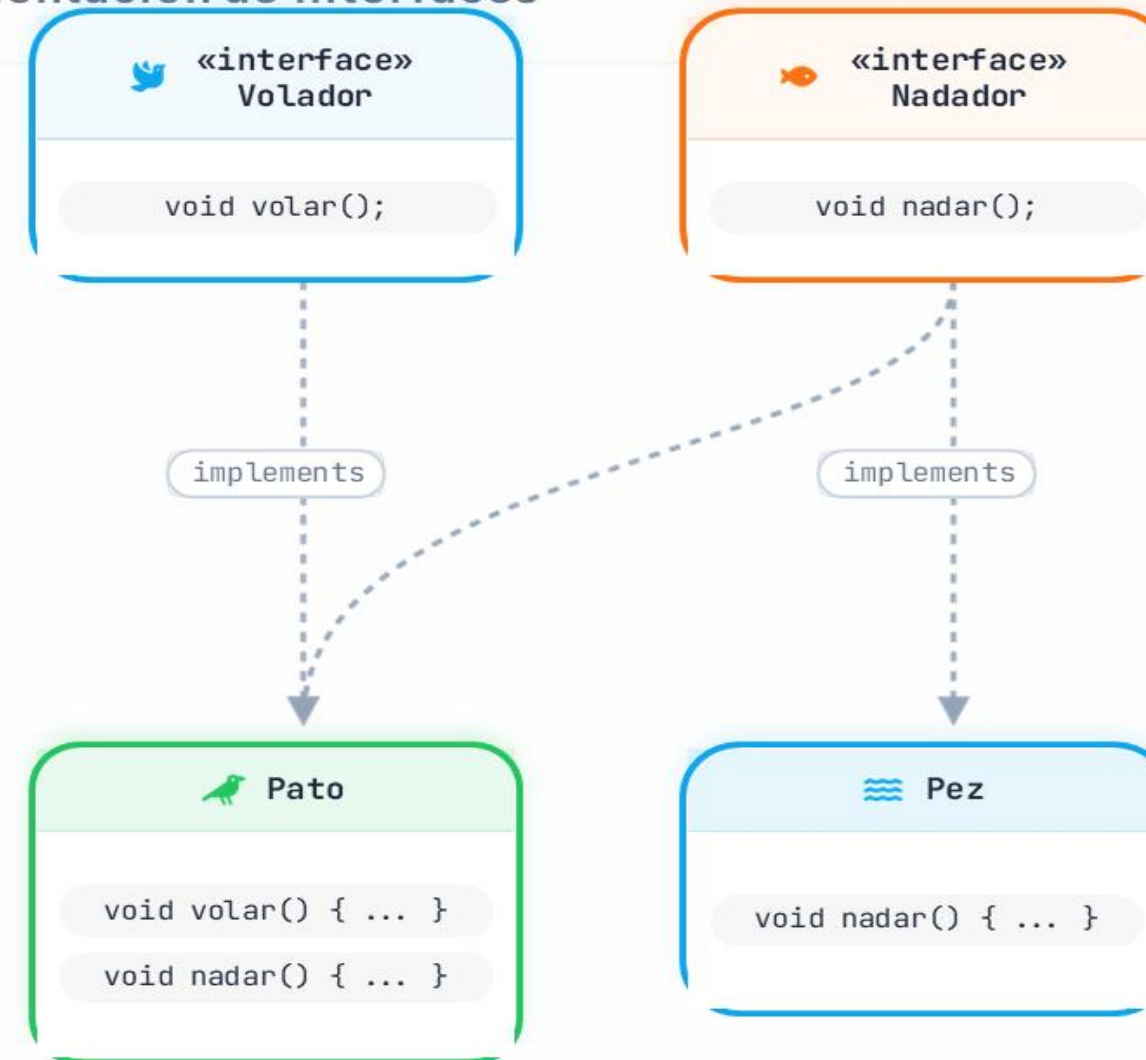
Capacidad: Nadador

Obliga a implementar nadar(). Una clase puede implementar múltiples interfaces.

Relación "Puede-hacer":

Un Pato **puede** volar y nadar, sin importar de qué clase base herede.

Implementación de Interfaces





Clase Abstracta vs Interfaz

Diferencias clave para el diseño flexible



Clase Abstracta

Usa clases abstractas cuando las clases compartan una relación fuerte y necesiten estado o lógica común.



Relación "Es-un"

Animal, Mamífero



Estado Central

Variables, constructores



Lógica Común

Métodos con cuerpo

Comparación Directa

Herencia

Simple
(extends)

Múltiple (implements)

Estado

Atributos de instancia

Solo constantes (static final)

Constructores

Sí (para subclases)

No

Métodos

Abstractos y concretos

Abstractos (y default)

Propósito

Identidad base (Qué es)

Capacidades (Qué hace)



Interfaz

Usa interfaces para definir contratos o capacidades que pueden ser implementadas por clases no relacionadas.



Relación "Puede-hacer"

Volador, Nadador



Múltiples Roles

Pato implementa ambos



Contrato Puro

Qué hacer, no cómo



Ejemplo base: abstract class Animal

La superclase abstracta define estado común y contratos

Clase Abstracta

La clase abstracta **Animal** centraliza el estado compartido y establece reglas obligatorias (métodos abstractos) que las subclases deben implementar.



Estado y Lógica Común

Define atributos como nombre y métodos concretos como dormir().



Métodos Abstractos

Obliga a implementar hacerSonido() en cada subclase específica.

Punto Clave:

 No se puede instanciar directamente

Estructura de la Clase Abstracta

abstract class Animal

Atributos (Estado Común)

```
A String nombre    # int edad
```

Método Concreto

```
void dormir()  
// Lógica compartida: System.out.println("Zzz");
```

Método Abstracto (Contrato)

```
abstract void hacerSonido();  
// Sin cuerpo, obliga a las subclases a definirlo
```





Subclases: Perro y Gato

Implementando comportamientos específicos de la clase abstracta



Implementación de Abstractos

Las subclases concretas **deben implementar** todos los métodos abstractos heredados o convertirse ellas mismas en abstractas.



Reutilización de Estado

Heredan los atributos nombre/edad y el método concreto dormir() de Animal.



Especialización Obligatoria

Deben proveer una implementación real para hacerSonido() con @Override.



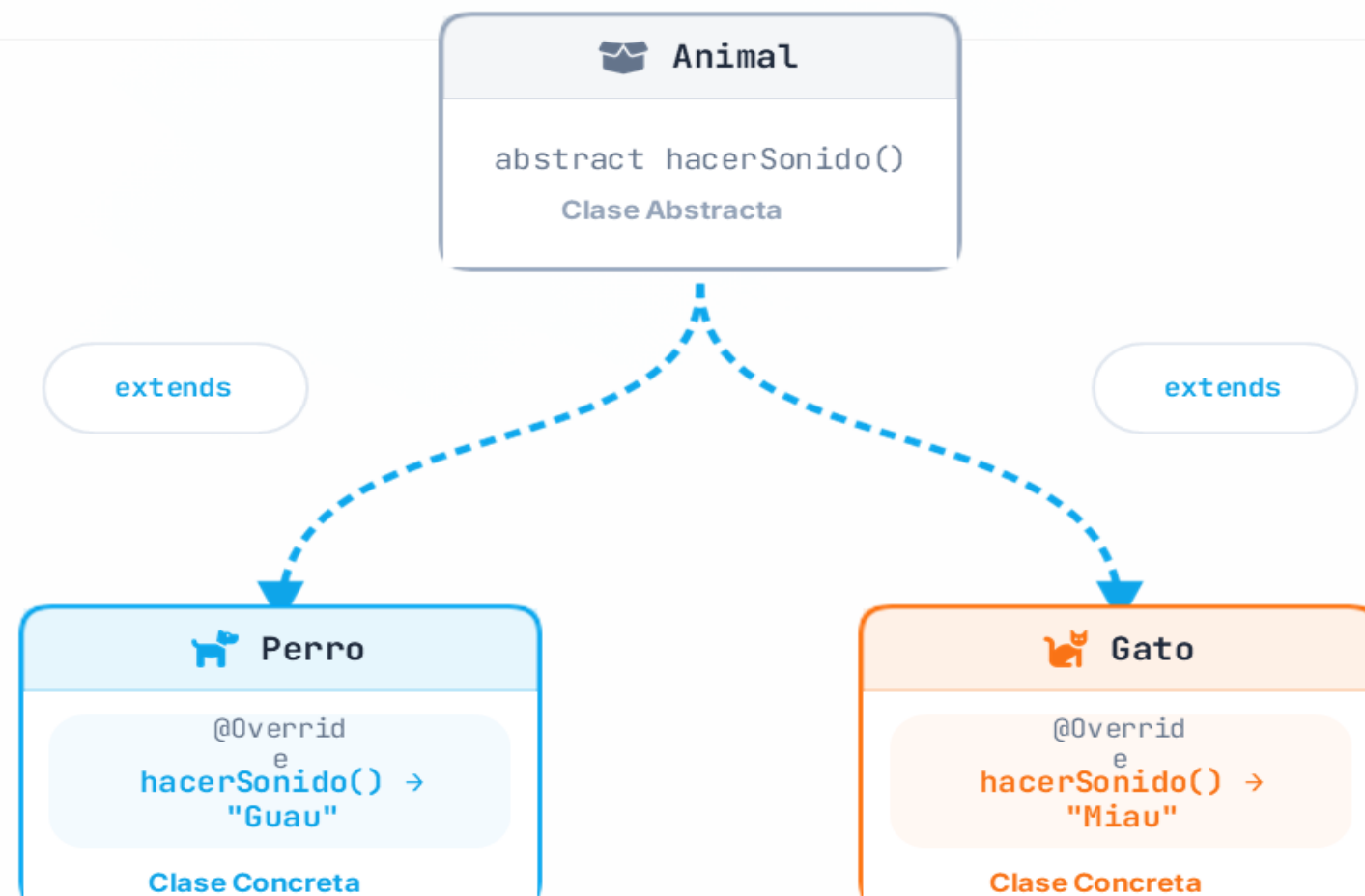
Clases Concretas

A diferencia de Animal, las clases Perro y Gato sí pueden ser instanciadas con new.

Resultado Final:

✓ Contrato cumplido con lógica específica

Estructura de Implementación



Clases Abstractas



Especialización



Interfaces en Acción

Modelando comportamientos transversales

Roles y Capacidades

Las interfaces permiten definir capacidades que pueden ser implementadas por cualquier clase, sin importar su lugar en la jerarquía de herencia.



Desacoplamiento

Separa qué hace una clase de cómo lo hace, evitando dependencias estrictas.



Múltiples Roles

En Java, una clase puede implementar múltiples interfaces simultáneamente.



Extensibilidad

Facilita añadir nuevas capacidades a clases existentes con un contrato claro.

Beneficio Clave:

- ✓ Diseño modular basado en capacidades

Contratos e Implementaciones



interface Volador

```
void volar();
```



interface Nadador

```
void nadar();
```



Clase Águila

```
implements Volador
public void volar() {
    print("Vuela alto");
}
```



Clase Pato

```
implements Volador, Nadador
public void volar() {
    print("Vuela"); }
public void nadar() {
    print("Nada"); }
```



Clase Pez

```
implements Nadador
public void nadar() {
    print("Nada rápido");
}
```

★ Pato implementa ambas capacidades





Implementación Múltiple con Pato

Combinando herencia e interfaces



Herencia + Múltiples Interfaces

En Java, una clase solo puede heredar de **una** clase abstracta, pero puede implementar **múltiples** interfaces.



Extiende Animal

Hereda estado (nombre, edad) y comportamiento base.



Implementa Volador

Adquiere la capacidad de volar.



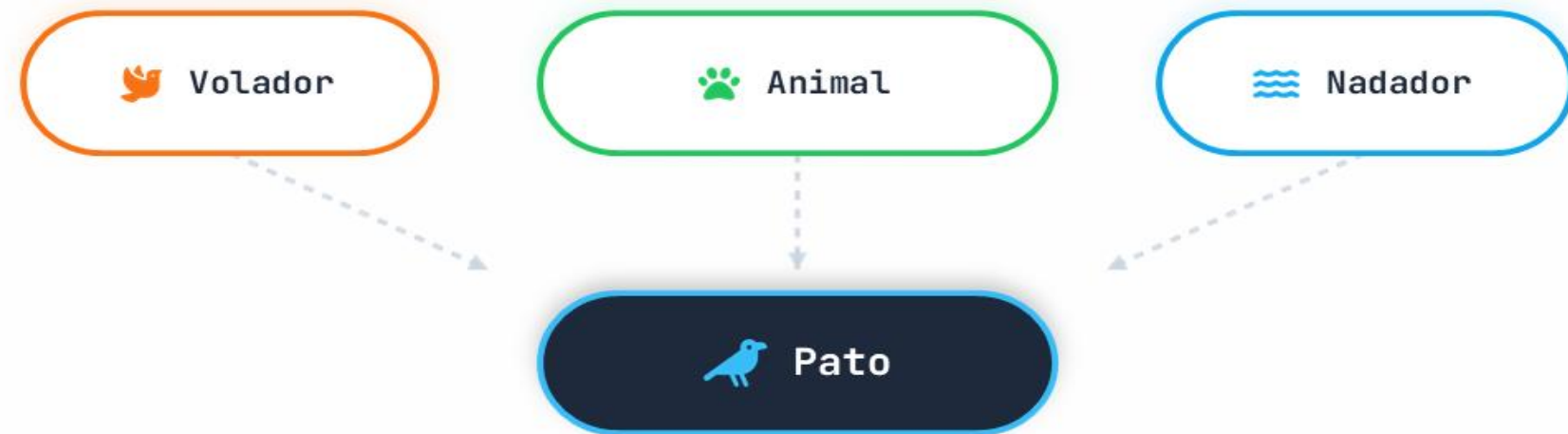
Implementa Nadador

Adquiere la capacidad de nadar.

Beneficio Clave:

- ✓ Diseño altamente modular y flexible

Estructura de la Clase Pato



```
public class Pato extends Animal implements Volador, Nadador {  
    @Override  
    public void  
    hacerSonido () { ... }  
    @Override  
    public void volar () { ... }  
    @Override  
    public void nadar() { ... } }
```



Clases Abstractas e Interfaces



Combinar Clase Abstracta e Interfaz

Arquitectura flexible: Estado base + Capacidades específicas

Patrón Recomendado

Combinar ambos mecanismos permite mantener un estado compartido (herencia) mientras se añaden comportamientos modulares (interfaces).



Clase Abstracta (Estado)

Aporta atributos comunes como nombre o edad, y lógica base compartida.



Interfaces (Capacidades)

Añaden roles específicos (Volador, Nadador) sin acoplar jerarquías completas.



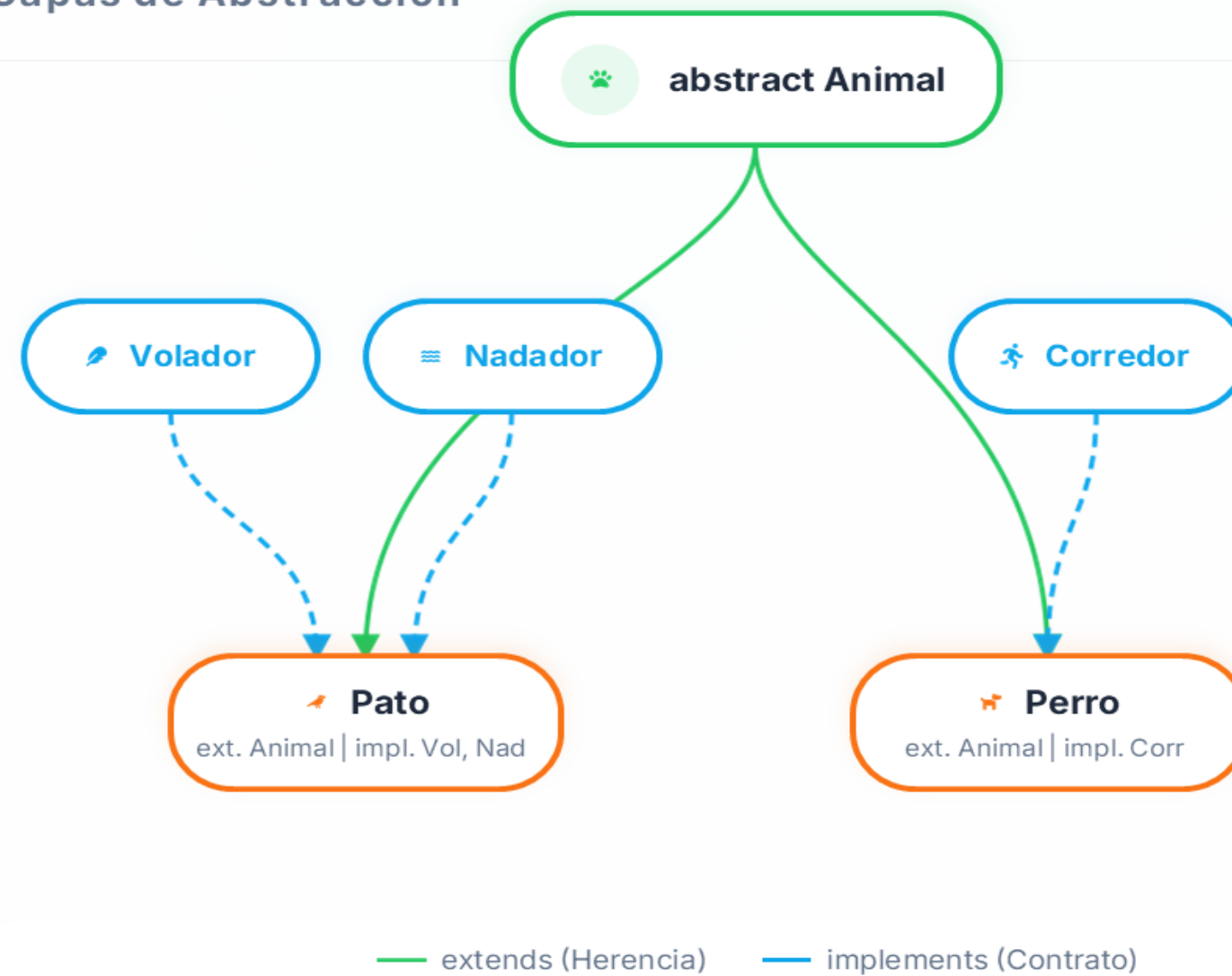
Máxima Flexibilidad

Permite que clases dispares compartan habilidades mediante polimorfismo.

Resultado:

- Diseño escalable y altamente reutilizable

Capas de Abstracción



Polimorfismo con Abstractas e Interfaces

Una misma referencia, múltiples comportamientos dinámicos

Poder Dinámico

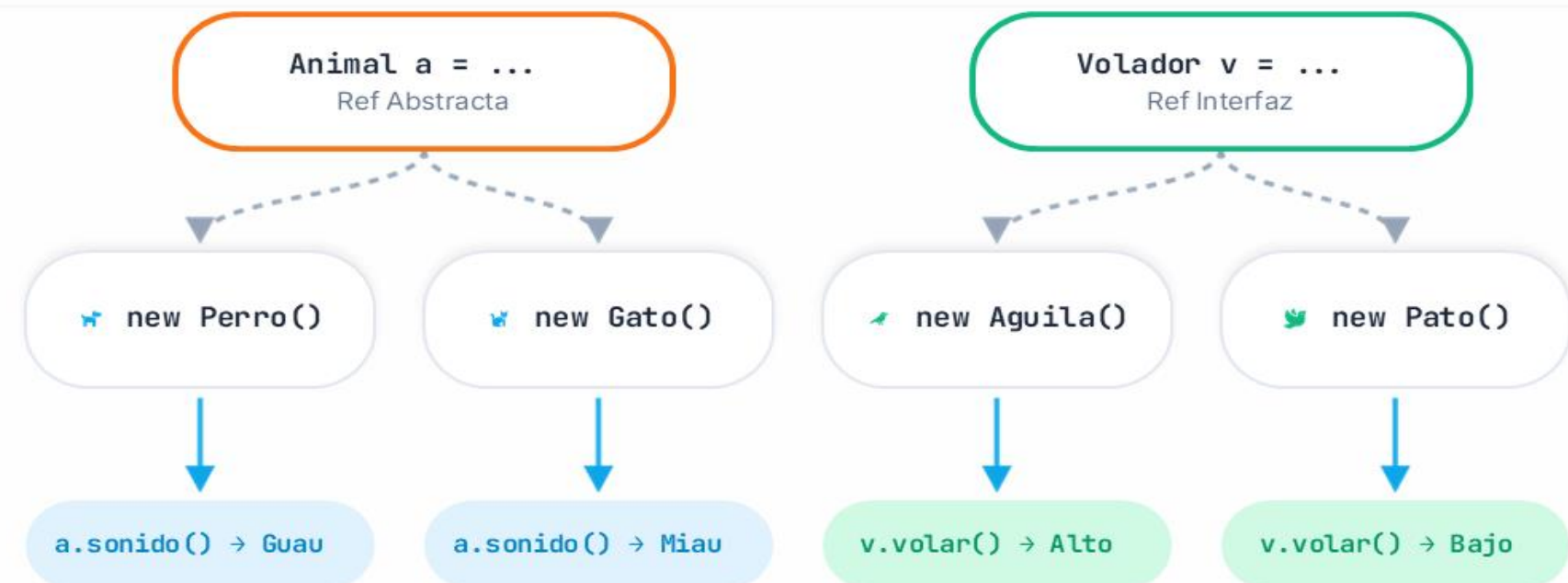
El polimorfismo permite que una referencia de **Clase Abstracta** o **Interfaz** apunte a cualquier objeto que la implemente o herede.

Esto desacopla el código: puedes procesar listas de `Animal` o `Volador` sin conocer las clases exactas.

Beneficios del Polimorfismo:

- ✓ Flexibilidad para cambiar implementaciones
- ✓ Código abierto a extensión, cerrado a modificación
- ✓ Tratamiento uniforme de colecciones

Resolución Dinámica de Referencias



Buenas Prácticas de Diseño

Reglas de oro para clases abstractas e interfaces

✓ Reglas de Diseño

Combinar correctamente clases abstractas e interfaces es la base de un código reutilizable y flexible.



Abstracta = Estado

Usa clases abstractas cuando necesites compartir atributos y estado base.



Interfaz = Capacidad

Usa interfaces para definir "lo que puede hacer" (ej. Volador, Nadador).



Composición > Herencia

Evita jerarquías profundas; prefiere componer objetos en lugar de heredar.



Nombres Claros

Nombra interfaces como adjetivos (-able, -dor) y clases como sustantivos.

Beneficio Clave: ✓ Buenas Prácticas
🔗 Diseño Sólido

Checklist Visual



Compartir atributos comunes

En clase abstracta (ej. nombre)



Heredar por conveniencia

Sin relación "Es un" real



Asignar roles transversales

Vía interfaces (ej. Nadador)



Interfaces enormes

Demasiados métodos obligatorios



Diseño Limpio y Escalable



Errores comunes y checklist

Puntos clave para diseñar arquitecturas limpias

⚠ Antipatrones a Evitar

Evita estos errores comunes para no crear jerarquías rígidas y sistemas acoplados en Java.



Instanciar Abstractas

Intentar crear objetos directamente con `new Animal()`.



Confundir "es-un" / "puede"

Usar herencia de clase para habilidades (ej: `class Volador` en vez de interface).



Interfaces Pesadas

Meter métodos no relacionados en una sola interfaz (viola ISP).



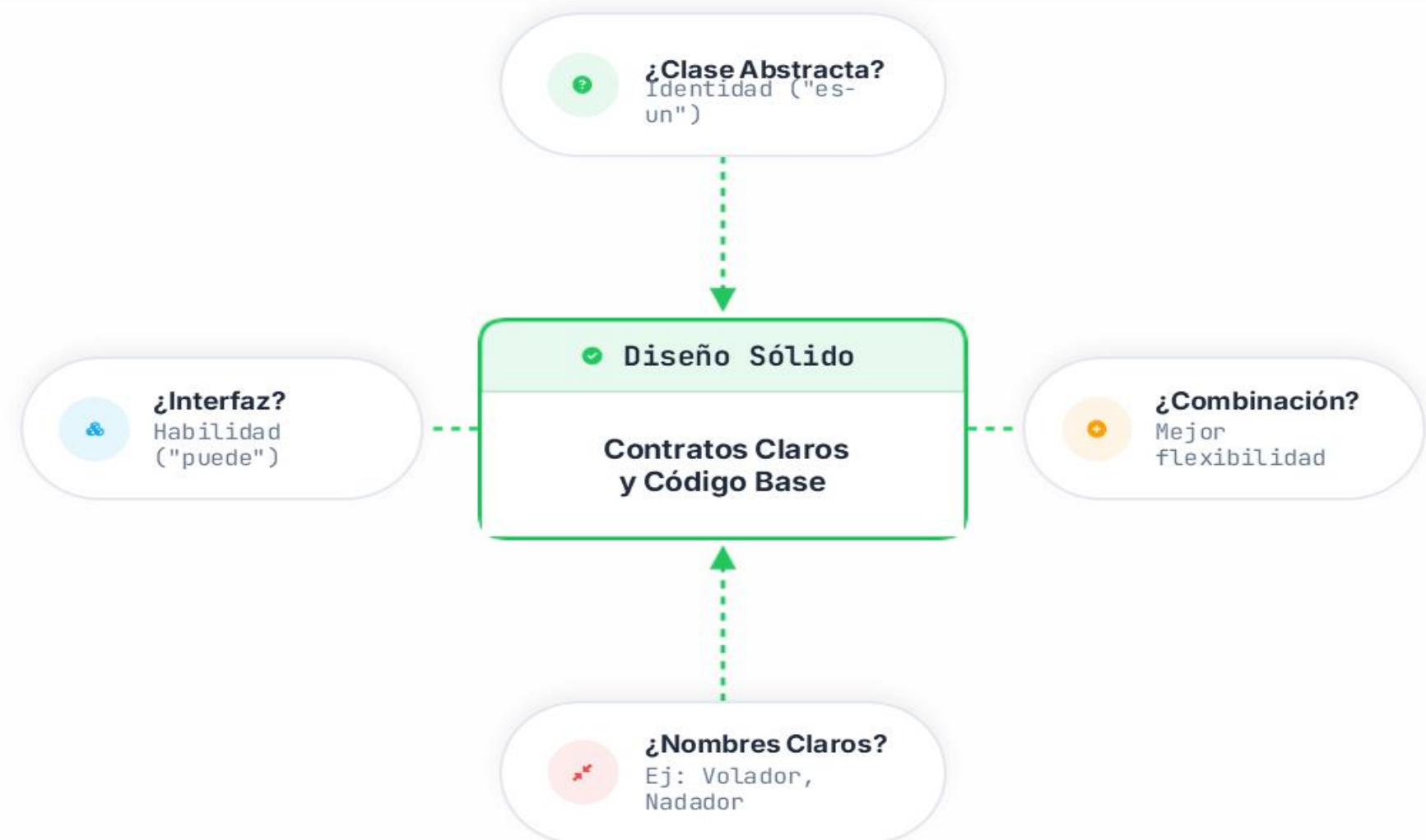
Abusar de Herencia

Crear cadenas profundas en vez de favorecer la composición de interfaces.

Resultado:

✓ **Arquitectura Flexible**

Checklist de Diseño



Consejo: Aplica este checklist antes de diseñar la jerarquía de tus clases.



Conclusiones y Próximos Pasos

Cierre de Clases Abstractas e Interfaces

✓ Síntesis Final

La combinación de **clases abstractas** para el estado base e **interfaces** para capacidades transversales crea diseños robustos y flexibles.

-  **Estado Común**
Usa clases abstractas para lógica y datos compartidos.
-  **Capacidades Flexibles**
Usa interfaces para contratos de comportamiento múltiple.
-  **Polimorfismo**
Programa hacia interfaces, no hacia implementaciones.

Resultado:

🏆 **Diseño Flexible y Reutilizable**

Arquitectura Combinada



→ Próximos Pasos Recomendados

- 1 Practicar con más animales
- 2 Patrones de Diseño
- 3 Colecciones en Java

Felicitaciones
¡Has finalizado el módulo!

DuocUC[®]



CERCANÍA. LIDERAZGO. FUTURO.

7
AÑOS
ACREDITADO

 Comisión Nacional
de Acreditación
CNA-Chile

NIVEL DE EXCELENCIA
HASTA OCTUBRE 2031

Docencia de pregrado / Gestión institucional / Aseguramiento interno de la
calidad / Vinculación con el Medio / Investigación, creación y/o innovación